

Theory and Practice of Unparsed Patterns for Metacompilation

Christian Rinderknecht Nic Volanschi

Konkuk University, Seoul,
23 May 2007

Foreword

I am [Christian Rinderknecht](#), an Assistant Professor at the department of Internet and Multimedia in the College of Telecommunications of Konkuk University.

My main research interests are

- ▶ the application of formal methods to software engineering,
- ▶ XML technologies as found in document processing and databases,
- ▶ protocol engineering of heterogeneous distributed systems.

The Ariadne thread is a focus on designing and making useful **tools** to support the engineer or the end-user, with a keen eye on formal modeling in order to add confidence about the implementations (by proving correctness, completeness etc.).

Foreword (cont'd)

My purpose today is to introduce myself and propose my collaboration on future projects in Mechanical Engineering. The topic of this talk is not directly related to Mechanical Engineering, but it illustrates how I am interested in the design and the making of tools for the engineer.

I will keep my presentation extremely simple, but questions are always welcome.

Executive summary

- ▶ I am a tool-maker kind of person
- ▶ with an unusual theoretical touch in order to be sure the hacks are sound;
- ▶ there are no dirty jobs: code writing, parsing, wrappers, system administration, middleware, tool and data integration etc. are all fine with me;
- ▶ Dr. Samy Missoum (University of Arizona) and I are willing to collaborate on a distributed MDO platform (perhaps based on Globus Toolkit).

The talk itself

This is work in progress.

The ideas presented here were initiated by Dr. Nic Volanschi

`http://mygcc.free.fr`

who implemented them in a working prototype for C++ and Ada
and published two papers about it.

My contribution here is the theory.

Plan

1. Compilation revisited
2. Metacompilation
3. Unparsed patterns
4. A glimpse at the theory

Software development tools

The software engineers have many kinds of tools at their disposal:

- ▶ text editors and graphical interfaces,
- ▶ Integrated Development Environments (IDE),
- ▶ compilers,
- ▶ profilers,
- ▶ debuggers,
- ▶ model checkers,
- ▶ code checkers, etc.

Checking programmes

Programme checking technology is now mature but not used on a large scale by developers. Why?

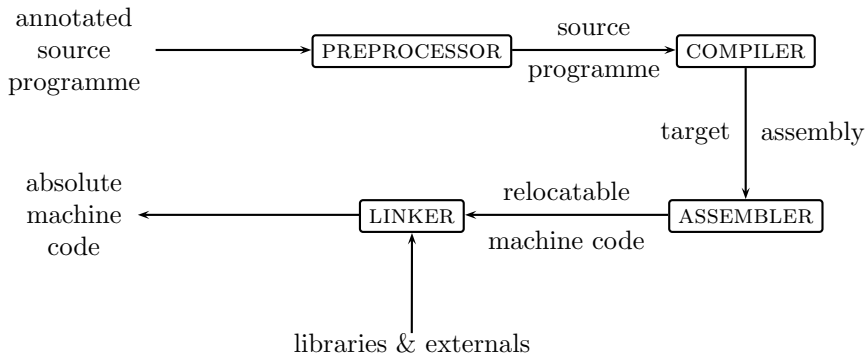
One of the reasons is the decoupling of checking tools from the development tools, in particular the compiler.

We explore here the integration of simple user-defined checks into the compiler, a technique known as [meta-compilation](#).

But first, we must recall what is the compilation chain.

Compilation chain

From an engineering point of view, the **compiler** is one link in a chain of tools:



The analysis-synthesis model of compilation

There are two parts to compilation: **analysis** and **synthesis**.

1. The analysis part breaks up the source programme into constituent pieces of an **intermediary representation** of the programme.
2. The synthesis part constructs the target programme from this intermediary representation.

We shall focus here on the analysis stage.

Analysis

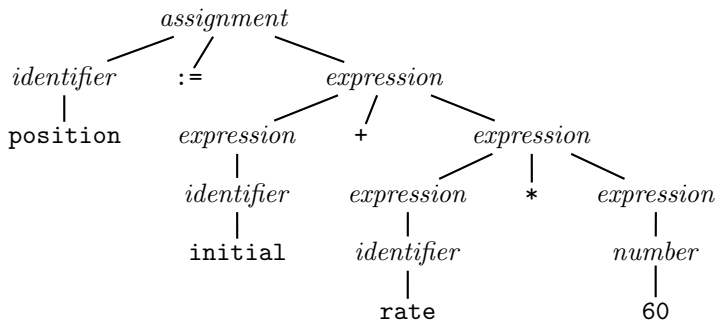
The analysis can itself be divided into three successive stages:

1. **linear analysis**, in which the stream of characters making up the source programme is read and grouped into **lexemes** that are sequences of characters having a collective meaning (sets of lexemes with a common interpretation are called **tokens**);
2. **hierarchical analysis**, in which tokens are grouped hierarchically into nested collections (**trees**) with a collective meaning;
3. **semantic analysis**, in which certain checks are performed to ensure the components of a programme fit together meaningfully.

We shall focus here on linear and hierarchical analysis.

Syntax analysis

Hierarchical analysis is called **parsing** or **syntax analysis**. It involves grouping the tokens of the source programme into grammatical phrases that are used by the compiler to synthesize output. Usually, the grammatical phrases of the source are represented by a **parse tree** such as:



Syntax analysis (continued)

In the expression

`initial + rate * 60`

the phrase

`rate * 60`

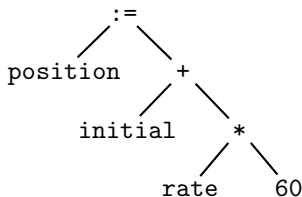
is a logical unit because the usual conventions of arithmetic expressions tell us that multiplication is performed prior to addition. Thus, because the expression

`initial + rate`

is followed by a `*`, it is **not** grouped into the same subtree.

Syntax analysis (continued)

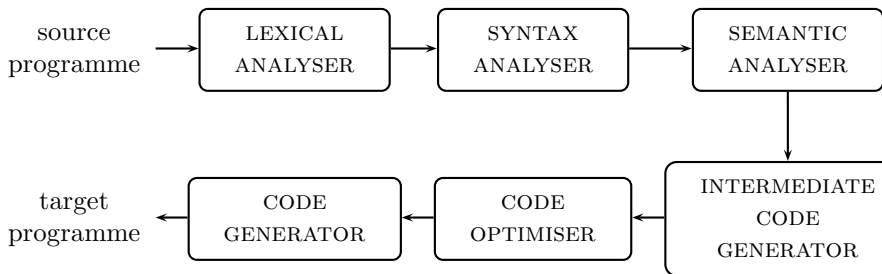
The parse tree page 12 describes the syntactic structure of an input. A more common *internal* representation of this syntactic structure is given by



This is an **abstract syntax tree** (or just **syntax tree**). It is a compressed version of the parse tree, where only the most important elements are retained for the semantic analysis.

The phases of a compiler

Conceptually, a compiler operates in **phases**, each of which transforms the programme from one representation to another. A typical decomposition of a compiler is as follows:



The first row makes up the analysis and the second the synthesis.

Plan

1. Compilation revisited
2. Metacompilation
3. Unparsed patterns
4. A glimpse at the theory

Metacompilation

Metacompilation is a technique that enables a compiler to take into account user-defined checks, instead of using a different tool, perhaps at run-time.

If the code added to the compiler to support this feature is efficient, checking can be done at every compilation.

Practically, it means that the programmer adds an option when calling the compiler on the command-line or clicks in an IDE.

Also, a user-defined file can be passed to the compiler.

Pattern matching

One kind of simple user-defined checks consists in writing a series of **patterns**, which are then matched against the code.

The purpose is to find whether some constructs are present in the code, in some contexts. The asserted presence or absence can be interpreted as an error or as a requirement.

For example, in C, one would like to check that a `malloc` is followed by a `free` before the function return. This supposes **control flow analysis**, which is already provided by the compiler internals.

Pattern matching (cont'd)

The simple approach is that the patterns are written in the same language as the source language.

But, in order to add expressiveness, **meta-variables** are added.

Meta-variables are variables that are expected to denote meaningful parts of the input programme. As such, they are not variables of the source language.

For instance,

```
buf = malloc(sizeof(int));  
%X = malloc(%Y);  
%L = %L -> next;
```

are patterns denoting C statements, where %X, %Y and %L are meta-variables.

Pattern matching (cont'd)

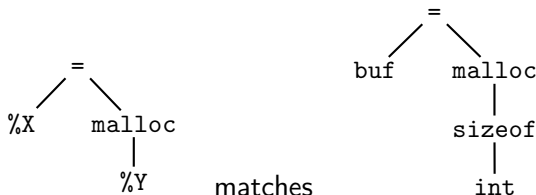
The usual process consists in

1. parsing the pattern into a pattern syntax tree,
2. matching it against the programme syntax tree (which is already available).

There is a match if the pattern syntax tree is a **subtree** of the programme syntax tree, assuming that meta-variables match any subtree.

Pattern matching (cont'd)

For example `%X = malloc(%Y);` matches
`buf = malloc(sizeof(int));` because



The result of a match is a set of **bindings** of meta-variables to syntax subtrees. Here, `%X` is bound to variable `buf` and `%Y` is bound to `sizeof(int)`.

The set of bindings is called a **substitution**. In general, applying the substitution to the pattern produces subtrees of the programme syntax tree.

Pattern matching (cont'd)

What is the point?

This approach is technically difficult to implement precisely because **the available built-in parser cannot handle the meta-variables**. Indeed, the patterns are not exactly expressed in the source language if they contain meta-variables.

When the parser has been built using standard tools (like Yacc), it means that the grammar has to be modified and two modes of the parser (one for parsing programmes, one for parsing patterns) have to be implemented on top of it.

Moreover, this technique has a **high cost in the worst case**.

Pattern matching (cont'd)

The prototype written by Dr. Nic Volanschi has been integrated to GCC and successfully found known bugs of a given version of the Linux kernel.

The purpose was to see if the unparsed patterns were enough expressive to catch a list of known bugs.

Now, the goal is to find new bugs and also to target Ada programmes, since GCC already includes GNAT (the GNU Ada compiler).

Plan

1. Compilation revisited
2. Metacompilation
3. Unparsed patterns
4. A glimpse at the theory

Unparsed patterns

What can we do?

Less.

Let us not parse the patterns and have a cost, in the worst case, which is proportional to the size of the syntax tree.

Of course, the challenge is to retain enough expressiveness in these **unparsed patterns**: they must be easy to work with and allow to filter as much as possible in order to satisfy the programmer.

The patterns keep the same syntax for the programmer, only the matching algorithm is different.

A glimpse at the theory

I made a **formal model** to express the new pattern matching and some property we expect it to have.

One of them is **soundness**. This concept requires two levels of abstraction, like a specification and an algorithm, or two algorithms, one being considered more intuitive than the other. Here, the intuitive algorithm captures the concept of matching for trees: given a pattern made of syntax trees, these trees are matched orderly against the programme tree (they share the same leaves). This is exactly like the matching of parsed patterns, except they contain no meta-variables. They are called **ground patterns**.

A glimpse at the theory (cont'd)

Then the soundness property states that,

- ▶ given a syntax tree and an unparsed pattern (possibly with meta-variables),
- ▶ if the matching of the unparsed pattern is successful,
- ▶ if the resulting substitution is applied back to the pattern,
- ▶ then the resulting ground pattern is matched successfully against the syntax tree.

In other words: the new pattern matching is sound with respect to the tree pattern matching.

A glimpse at the theory (cont'd)

Another interesting property is the reverse of soundness:

completeness.

It means that all matches found by the tree matching algorithm are found by the new pattern matching. (Not proved yet.)

Another important task is to compute the **worst-case complexity**, to make sure the algorithm is efficient.

A glimpse at the theory (cont'd)

The logical framework used to define the algorithm is **inference rules**. It is an axiomatisation of the theory which allows both to define an algorithm and prove properties on it.

$$\frac{t \in \mathcal{H} \quad \hat{p} \sqsubseteq [t]}{\hat{p} \sqsubseteq t} \text{ ONE}$$

$$[] \sqsubseteq [] \quad \text{EMP}$$

$$\frac{t \in \mathcal{H} \quad \hat{p} \sqsubseteq f}{[t \mid \hat{p}] \sqsubseteq [t \mid f]} \text{ EQ}$$

$$\frac{[t \mid \hat{p}] \sqsubseteq t_c \cdot f}{[t \mid \hat{p}] \sqsubseteq [c(t_c) \mid f]} \text{ SUB}$$